

NODE JS

DATE
PAGE 75

Back End

to initialize a backend Project Command.

```
npm init
```

-y to avoid
setup questions

to install node Packages

```
npm install. package_name.
```

Common JS Vs Ecma Script. Modules.

Common JS

Ecma Script **ES6**

(Modern)

"type": "commonjs"

"type": "module"

* importing Modules.

```
const http = require("node:  
http")
```

```
import http from "http"
```

```
const fs = require("fs")
```

imports & exports.

```
let a = 10  
module.exports = a
```

```
export const a = 1  
export default a  
const obj = {
```

```
const a = require("filename.js")
```

```
  a: 5,  
  y: 2  
}
```

```
import {a} from './filename.js'  
import any-name + from " " ↓ "
```

FS Module

const fs = require("fs") (common.js)
or.

import fs from "fs" (ES)

Writing to file.

fs.writeFileSync("filename", "Content")
Sync, creates files if not already there.

```
fs.writeFile("filename", "content", () => {  
  // handler fn.  
})
```

async ~~fs.writeFile~~

Reading from file.

~~fs.readFile("filename", (error, data) => {
 // code
})~~

```
fs.readFile("Path", (error, data) => {  
  // code.  
  console.log(error, data.toString())  
})
```

```
fs.appendFile("Path", "Content", (e, d) => {  
  // handler  
})
```

↑ error ↑ data.

fs/promises. (modern form of fs module)

```
import fs from "fs/promises"
```

```
const a = await fs.readFile("path").toString()
```

Similarly,

```
  .writeFile("Path", "Content")
```

```
  fs .appendFile("path", "Content")
```

#

Path.js

```
import path from "path" ES6
```

```
path.extname("any-path")
```

returns extension of the file @ "any-path"

```
path.dirname("any-path")
```

returns parent dir.

```
path.basename("any-path")
```

returns the file name.

```
path.join("path1", "path2")
```

joins & returns two paths.

https://google.com/blog?region=in & mode=dark
slug queries
req.params req.queries
DATE PAGE 78

Introduction to Express JS

```
import express from 'express'
const app = express()
const port = 3000
app.get('/', (req, res) => {
  res.send("Hello World")
})
```

```
// app.get or app.post or app.put or app.delete
app.listen(port, () => {
  console.log('port open, listening started')
})
```

Using request.params

```
app.get('/:slug/:slug1', (req, res) => {
  // code using ${req.params.slug} and ${req.params.slug1}
  res.send('hello ${req.params.slug} and ${req.params.slug1}')
})
```

static files.

```
app.use(express.static('public'))
```

to make files inside 'public' folder, public so that anyone can view using request

Response, Request & Routers

```
app.get("endpoint", Handler).put(...).post(...).delete(...)
```

this is chaining.

to render HTML Files using requests.

~~res.sendFile (Path to the HTML file.html)~~

res.sendFile ('path/to/HTMLfile.html', {root: __dirname})

~~Response Methods~~

Routers in ExpressJS

./routes/About

./server.js

```
import express from 'express'
const router = express.Router()
const router = express.Router()
```

```
import express from 'express'
import about from './routes/About'
const app = express()
const Port = 3000
```

```
router.get ---
// all the routes
// for About Page.
```

```
app.use("/about", about);
```

```
export default router;
```

now for all the routes
via this will be in
'./routes/About'

Middlewares in EJS

Syntax

```
app.use ( (req, res, next) => {  
  // code.  
  next()  
})
```

types of Middlewares

① Application lvl Middleware

```
app.use ( (req, res, next) => {  
  // code.  
  next()  
})
```

used for main application.

② Router lvl Middleware

```
router.use ( (req, res, next) => {  
  // code.  
  next()  
})
```

used for specific Routers.

③ Error handling Middleware.

```
app.use((err, req, res, next) => {  
  console.error(err.stack);  
  res.status(500).send('Something went  
  wrong!')  
})
```

④ Builtin Middlewares

`express.static` (serves static assets such as HTML files, imgs, and so on.)

- `json` (parses incoming requests with JSON payloads)
- `urlencoded` (Parses incoming requests with URL encoded payloads)

Embedded JS EJS Template Engines in Express

① Setting engine in Express

```
app.set('view engine', 'ejs');
```

② Directory structure.

```
views
├── index.ejs
└── main.js
```

(This is the template HTML file but now .ejs to be used with ejs)

③ Syntax.

→ in main.js

```
app.get('/', (req, res) => {
  // code
```

```
  res.render("index", is object)
```

```
  { name: "Atharva", branch: "cs" }
```

})

→ in index.ejs

```
<h1> <% = name %> </h1>
```

include Components (Partials)

```
<%- include('navbar'); %>
```

↓
navbar.ejs

Mongoose

Model

- * This is used to ~~make~~ make a schema for db.
- * Mongo-db doesn't have a ~~def~~ specified schema by default.

Syntax:

creation of Schema (with mongoose)

models
↳ db.js

```
import mongoose from 'mongoose'
const dbSchema = new mongoose.Schema({
  t-name: String,
  and all other fields with their
  respective datatype
});
```

```
export const dbmodel = mongoose.model(
  'dbmodel', dbSchema)
```

* Flags for any Schema record

```
f name: { type: String, required: true, default: "Hey!" }
```

use of schema to store data to the DB.

~~ie~~

~~main.js~~

```
import { dbmodel } from '../models/db.js'
import mongoose from 'mongoose'
// mongoose standard call
let db = await mongoose.connect('db url')

const dbmodel = {
  Record object
}
```

main.js

```
import mongoose from 'mongoose'
import { dbmodel } from '../models/db.js'
```

```
let db = await mongoose.connect('db url')
```

```
const db_obj = new dbmodel({
  name: "Atharva",
  all other data.
```

```
})
```

```
db_obj.save() // stores data to db.
```

~~Find One~~

Search from DB.

~~also~~
const data. = await model.findOne({
 "email": "example@abc.com"
})

- returns complete document as object if the doc is available.
- returns 1st doc if multiple matching doc are available

Authentication

DATE
PAGE

89

bcrypt.js & JWT JSONWebToken
Modern Stateless Auth Token.

hashing

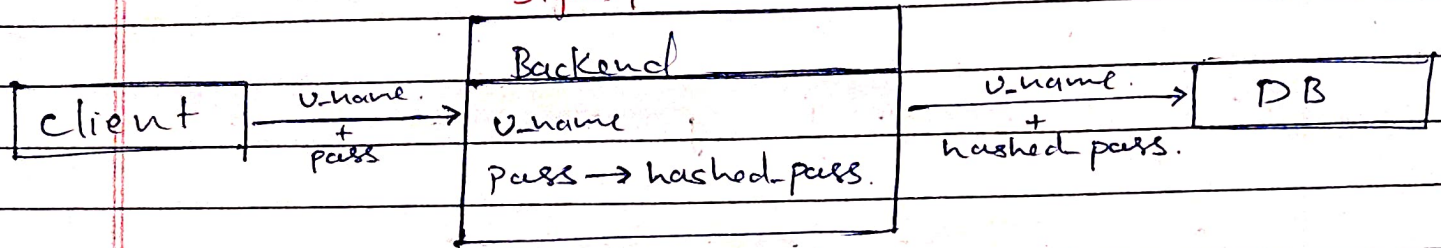
* hash: its a 10 character string.

* hashing: one way i.e.:-

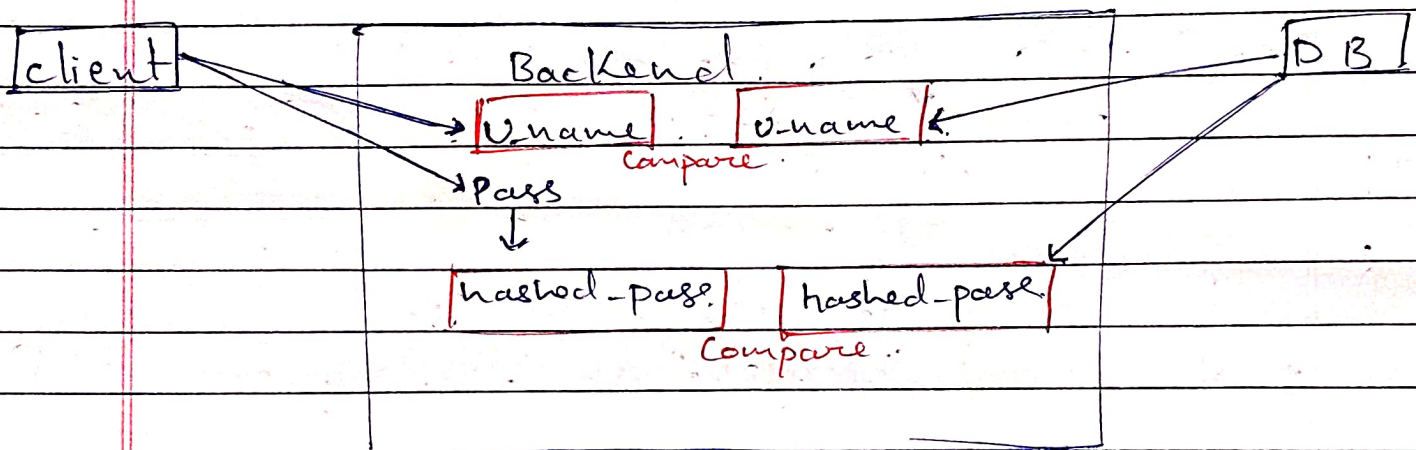
password $\rightarrow$ hashed string ✓

X password $\leftarrow$ X hashed string X

SignUp



Login



Limitation: Rainbow tables can be Prepared.

Table contains all ~~passwords~~ common password combinations with their respective hash

to overcome this we use salt

Salt: a random characters ~~add~~ 'example: @!&' are added to password b4 hashing and this salt is saved separately @ DB

Papper: its a random string just like salt but common for all users i.e., stored at Backend server

Tailwind. CSS

→ used for Rapid development

→ play.tailwindcss.com

BcryptJS Continued

npm i bcryptjs jsonwebtoken

import bcrypt from 'bcryptjs'

~~const salt = bcrypt.genSalt(10)~~
~~const pass = bcrypt.hash(req.body.pass, salt)~~
 await

Creation of Hashed Pass & JWT

const salt = await bcrypt.genSalt(10)

const pass = await bcrypt.hash(req.body.pass, salt);

→ this is hashed password.

import jwt from 'jsonwebtoken'

const jwtData = jwt.sign(data, JWT_SECRET)

↓
this is encoded.

JWT auth token.

↓
user data.

{

id: retrieved from DB

email:

}

res.send(jwtData)

Email Validator.

npm i express-validator.

```
import { body, validationResult } from express-validator
```

applies checks on req.body

```
body('email').isEmail()
```

```
body('email').normalizeEmail()
```

```
body('password').exists()
```

```
const errors = validationResult(req);  
return errors;
```

Comparing & Verifying Password using bcrypt.

```
const isPassValid = bcrypt.compare(password, db.password);
```

not hashed password (received from client) ↓

↑

hashed password (received from DB)

await

(return in boolean)

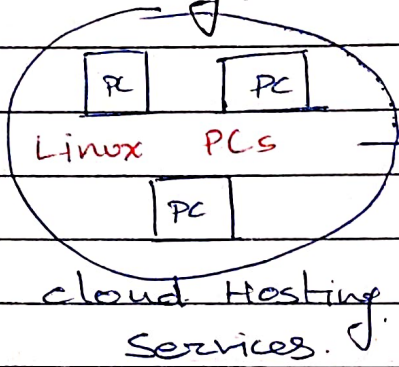
JWT Verify.

```
const data = jwt.verify(token, JWT_SECRET)
```

↓

Payload received from token.

Hosting.



1) IP

2) Username.

3) Password.

@ Terminal using
SSH.

File Zilla file transfer service.

Windows commands for hosting.

ssh root @ <ip-address>

for connecting to the server.

Types of Hosting.

- ① Shared. (a user of a Multi user VM).
- ② Dedicated. (full VM)
- ③ Managed. (Dashboard cused for wordpress)

Environment Variables.

~~npm install dotenv~~

① npm install dotenv

② make a '.env' file and store all secrets in it

③ import 'dotenv/config'
console.log(process.env)

Web server. (apache2) or (NGINX)

apt install apache2

VPS Hosting

Virtual Private Server (cloud server)

React `npm create-react-app foldername`

Vite `npm create vite@latest`

Components & Props

Basic syntax:-

(Components / navbar.jsx)

```
import React from "react"
import "./navbar.css" // for styling
```

```
const navbar = (props) => {
  return (
    <nav>
      <div> <h1> { props.heading } </h1>
    </nav>
  )
}
```

Object.
↳ props (dynamic variables which can be added)
only one tag can be used in return.

export default ~~const~~ navbar.